



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

Course Code: CSE 402.

Course Title: Operating System Lab.

Experiment No: 5

Experiment Title: Implementation of Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm

Date of Submission : 29-08-2026

Submitted By:

Name : Boishakhi Basu.

Student Id : 22201173

Semester : 4th Year 1st Semester

Section : C-2

Submitted To
Atia Rahman Orthi
Lecturer , UAP

Problem Statement:

Given the following processes with their **Arrival Time (AT)** and **Burst Time (BT)**:

Process	AT	BT
P1	4	2
P2	2	2
P3	1	3
P4	0	6
P5	3	1

Implement the **Preemptive Shortest Job First (SJF)** CPU Scheduling Algorithm with a **Time Quantum (TQ)** of 2 units.

For each process, calculate **Completion Time (CT)**, **Turnaround Time (TAT)**, and **Waiting Time (WT)**. Also, calculate the **Average Turnaround Time (Average TAT)** and **Average Waiting Time (Average WT)**.

Finally, represent the **process execution sequence** and display the scheduling results for all processes.

Source Code:

```
process = ["p1", "p2", "p3", "p4", "p5"]
at = [4, 2, 1, 0, 3]
bt = [2, 2, 3, 6, 1]

n = len(process)

ct = [0] * n
wt = [0] * n
done = [0] * n
tat = [0] * n
```

```
tq = 2
remaining = bt.copy()
time = 0
completed = 0

while completed < n:

    x = -1

    for i in range(n):

        if at[i] <= time and remaining[i] > 0:

            if x == -1 or remaining[i] < remaining[x]:
                x = i

            if x == -1:
                time += 1

            else:
                if remaining[x] > tq:
                    time += tq
                    remaining[x] -= tq

                else:
                    time += remaining[x]
                    remaining[x] = 0
                    ct[x] = time
                    completed += 1

    for i in range(n):
        tat[i] = ct[i] - at[i]
        wt[i] = tat[i] - bt[i]

print("Process\tAT\tBT\tCT\tTAT\tWT")
```

```

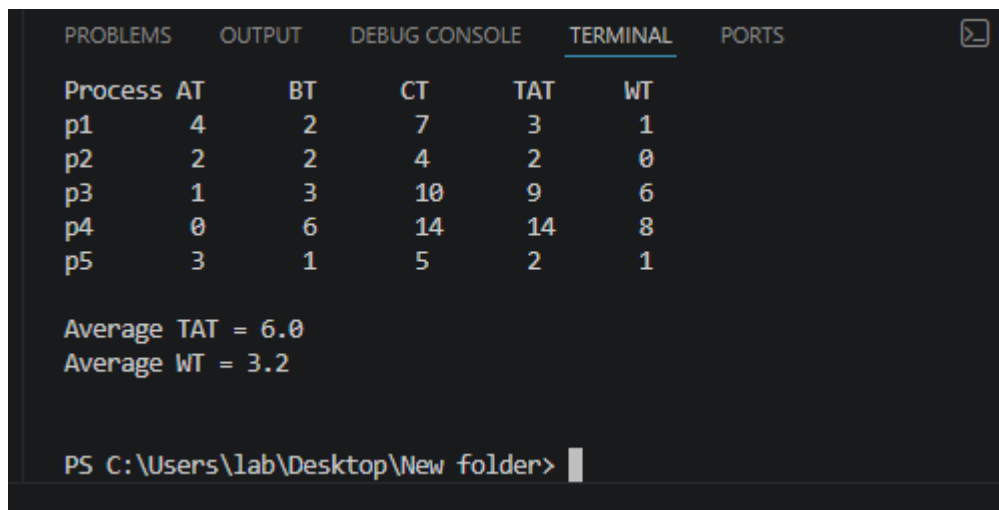
for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i], "\t", tat[i], "\t", wt[i])

print("\nAverage TAT =", sum(tat)/n)
print("Average WT =", sum(wt)/n)

print("\n")

```

Output:



Process	AT	BT	CT	TAT	WT
p1	4	2	7	3	1
p2	2	2	4	2	0
p3	1	3	10	9	6
p4	0	6	14	14	8
p5	3	1	5	2	1

Average TAT = 6.0
Average WT = 3.2

PS C:\Users\lab\Desktop\New folder>

Discussion:

In this experiment, the **Preemptive Shortest Job First (SJF)** scheduling algorithm was implemented with a Time Quantum of **2 units**. The algorithm selects the process with the shortest remaining burst time from the available processes. After executing a process, the scheduler checks again whether another process with a shorter remaining time should be selected.

The results can also be compared with **Non-Preemptive SJF**. In **Preemptive SJF**, a running process can be interrupted when another process with a shorter remaining execution time becomes available. Therefore, shorter processes can receive CPU time earlier. In contrast, **Non-Preemptive SJF** does not interrupt a

process once it starts execution; the selected process continues until it is completely finished.

For the given processes, the performance of Preemptive and Non-Preemptive SJF can be compared using **Average Turnaround Time (TAT)** and **Average Waiting Time (WT)**. The algorithm with lower Average TAT and Average WT is considered better for the given process set.

Conclusion:

The experiment demonstrates the working principle of the **Preemptive SJF scheduling algorithm** and its difference from the **Non-Preemptive SJF algorithm**. Preemptive SJF allows the CPU to switch to a newly arrived process with a shorter remaining burst time, whereas Non-Preemptive SJF allows a selected process to run until completion.

Generally, **Preemptive SJF can provide better responsiveness for shorter processes** because they do not have to wait for a longer running process to finish. However, the better algorithm should be determined by comparing the calculated **Average TAT and Average WT** from both implementations. For the given dataset, the algorithm that produces lower Average TAT and Average WT can be considered the better-performing scheduling algorithm.

Therefore, the comparison shows that Preemptive and Non-Preemptive SJF have different advantages, and the final decision depends on the calculated performance results for the given processes.

GitHub:

<https://github.com/boishakhi73/CSE-402-Operating-System-Lab>